CREST Gold Report

Title: Behavioural Fingerprinting from Keystroke Dynamics

Author	Devadit Jain
Field	Machine Learning, Behavioural Biometrics and Cybersecurity
Type	Research and Build
Dates	November 2025 – July 2026
Video Link	https://drive.google.com/file/d/1tmuonxF6AnwRGdd49PdMm7Z5HObn92vw/view?usp=sharing
Website Link	typing-sanctuary.onrender.com

Abstract

My project examines whether a person can be identified from the rhythm of their typing, and whether a learned model can match the hand-built methods that have defined the task since the 2000s. I encoded keystroke timing into a fixed-length numerical fingerprint and verified each new sample against a user's enrolled fingerprints using a set of classical distance statistics computed within the learned space. I evaluated the system on a public dataset of 51 subjects typing a single fixed password, scoring it only on subjects withheld from training. It attained an equal error rate of 14.2%, short of the 9.6% reported by the strongest method in the 2009 study that established the dataset.

Contents

1. Introduction
2. Background and related work
3. Method
4. Evaluation design
5. Results
6. Discussion
7. Ethics and responsible use
8. Reflection and future work
9. Acknowledgements, AI use, References and Appendices

1. Introduction

Someone recently broke into my Gmail account using my password. The system raised no alarm, and why should it have? The password was correct, so as far as the system was concerned, the person typing it *was* me. This is the quiet flaw beneath every password ever made: it checks what you *know*, not who you *are*. Steal the secret, and you inherit the identity.

This is not a rare problem. Verizon's 2024 security report [12] attributed about 88% of basic web-application attacks to stolen passwords, most of them automated: attackers take millions of leaked passwords from one site and try them against the login pages of every other, on the assumption that people reuse them. Usually they do. Two-factor codes help, but they interrupt the user at every login, and anything that interrupts is eventually switched off.

So I asked a different question. What if the second check asked nothing of the user at all, and simply watched *how* they already type? Typing has a rhythm. The small pauses, the keys held a fraction longer than others, the slight hesitation before a capital: all of it is as personal as an accent, and it comes from motor habits that are hard to fake on purpose. The idea is old. A century ago, telegraph operators could recognise one another by the rhythm of their tapping. What is new is that computers can now do this from any text a person types, instead of one memorised phrase.

My aim was to test whether a computer can verify a person's identity from typing rhythm alone, and how close it can come to the best published benchmark. I broke this into five objectives:

1. **A working fingerprint.** Build a network that turns a burst of typing into a fixed-size list of numbers, on an ordinary laptop. *Done when* one sample fingerprints in under a millisecond on a CPU.
2. **Fingerprints that cluster by person.** Train it so the same person's samples land close together and different people land far apart. *Done when* two samples from the same held-out person are reliably closer than two from different people.
3. **A clean score.** Test it only on people it has never seen, with the code refusing to grade itself on anyone it studied. *Done when* a runtime check makes a training/test overlap impossible.
4. **A fair comparison built in.** Add a classical statistical decision-maker on top, so I can weigh a simple method against an elaborate one from the same run. *Done when* both scorers report error rates from identical splits.
5. **Reproducible and fail-safe.** Make the whole system rerun from one command, and fail safe when live, so any breakage asks for another factor rather than letting someone in. *Done when* fixed seeds and a pinned dataset reproduce every number.

How the project ran - The build ran across seven months, about 70-80 hours in total. I have split it into six stages below that can be verified from the commit history.

Stage	Planned	Actual	Slip	Focus and outcome
1. Typing platform and keystroke-capture engine	Nov 2025	Nov 2025	on time	The full-stack site that records every keystroke; it became the project's data source.
2. User accounts and secure authentication	Dec-Jan 2026	Feb 2026	+4 weeks	Google logins and session handling, the gate in front of the biometric layer; OAuth and session bugs cost four weeks here.
3. First biometric engine and the research/serving architecture	Feb 2026	Mar 2026	+4 weeks	Two statistical keystroke recognisers (v1/v2), and the clean split between research code and the live product.
4. Deep metric-learning model: encoder, training, verifier	Jun 2026	Jun 2026	on time	The learned 128-number fingerprint, the triplet-loss training pipeline and the three-distance verifier.
5. Open-set evaluation harness and validation	Jun 2026	Jun 2026	on time	The open-set test protocol, the mid-June rebuild after the training/test leak, and the checks: 14 seeds, nested validation, a Transformer baseline.
6. Analysis, figures, and the scientific write-up	Jun 2026	Jun–Jul 2026	+2 weeks	The results and figures, and this report.

Stages 2 and 3 each ran about a month late: authentication took longer than expected, and so did the first biometric engine. That was okay though, because I'd planned to take a break in April and May for my final exams and the SAT. I restarted work in June which initially went to plan. However, I made some changes to the project while writing the final report which ended up taking a little longer than expected and I finished the report about 2 weeks into July.

2. Background and related work

Recognising someone by their typing is a behavioural biometric: it measures how a person acts, like a signature or a gait, rather than what the body is, like a fingerprint or an iris. The raw signal is only timing, how long each key is held and the gaps between keys.

The problem has an easy version and a hard one. If everyone types the same phrase, matching keystrokes can be compared directly. If a person can type anything, the rhythm has to be read without relying on the words. Always-on security needs the hard version, and the hard version is within reach: in 2013, Frank and colleagues showed with 'Touchalytics' [10] that even the way a person swipes a touchscreen carries enough identity to verify them continuously. I initially worked on the easy version, which has a clean public benchmark, while building the machinery the hard version would later need.

Scoring such a system needs care, because it can make two opposite errors: admitting an impostor and rejecting the genuine user. Tighten the threshold and it rejects real users; loosen it and impostors slip through. Since the threshold can sit anywhere, a single "accuracy" figure would be meaningless. The standard summary is the equal error rate (EER), the setting at which both errors are equally likely; at 10%, roughly one impostor in ten is admitted and one genuine user in ten is turned away.

Two studies define the limits of the field, and they have never been compared directly. In 2009, Killourhy and Maxion [1] built the dataset I used, 51 people typing the password ".tie5Roanl" 400 times each, and tested 14 hand-designed detection methods against it; the best achieved an equal error rate of 9.6%. In 2021, a system called TypeNet [2] reached 2.2%, but under entirely different conditions: it learned from 136 million keystrokes collected from roughly 168,000 people typing freely on the web. A recent review in ACM Computing Surveys [9] traces the field's movement from the first kind of system to the second. What no study had tested is the point where the two meet: whether a learned model, applied to the small fixed-text dataset of 2009, can beat the hand-designed methods on their own terms when judged only on people it has never seen – which is exactly what I set out to answer with my project.

3. Method

There were three primary ways to build this, and I weighed them before choosing:

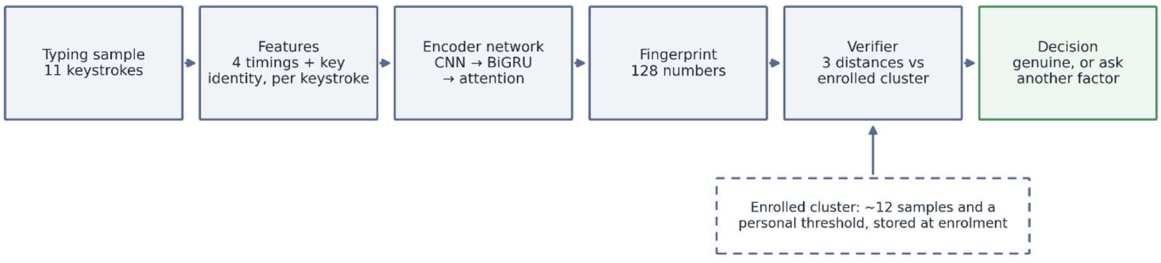
- **Pure statistics.** Compare each new sample against a hand-built statistical profile of the user. Simple, well understood, and the basis of the 2009 benchmark, but tied to one fixed phrase and dependent on a human to choose which features matter.
- **A standard classifier.** Train a network to sort typing into "user 1, user 2, …". Accurate, but it would have to be retrained from scratch every time a new user registers, which is unworkable for a real product.

- **A learned fingerprint** *(what I chose)*. Train the network to *place* each sample in a space, so that a new user simply adds a few of their own points. No retraining, and not tied to one phrase.

Approach	Cost of a new user signing up	Tied to one fixed phrase?	Feasible on a laptop?	Main risk
Pure statistics *(the 2009 benchmark)*	Cheap: build one statistical profile	**Yes**, phrase-locked	Yes	A human must hand-pick the features; cannot extend to free typing
Standard classifier	**Ruinous**: retrain the whole network per signup	No	Trains on a laptop, but retraining kills it in production	Unusable as a live service; retraining latency grows with every user
Learned fingerprint *(chosen)*	Cheap: record ~12 samples, no retraining	No	Yes (~23 min to train once, <1 ms per check)	Needs enough people up front to learn a good space

I ruled out the classifier despite its accuracy on paper: a network that must be retrained every time someone registers cannot serve a real login system. Of the remaining two, the learned fingerprint is not tied to a single phrase, so I chose it and ran the classical statistics inside its learned space. That hybrid is the central idea of my project.

Figure 1: the method, end to end.

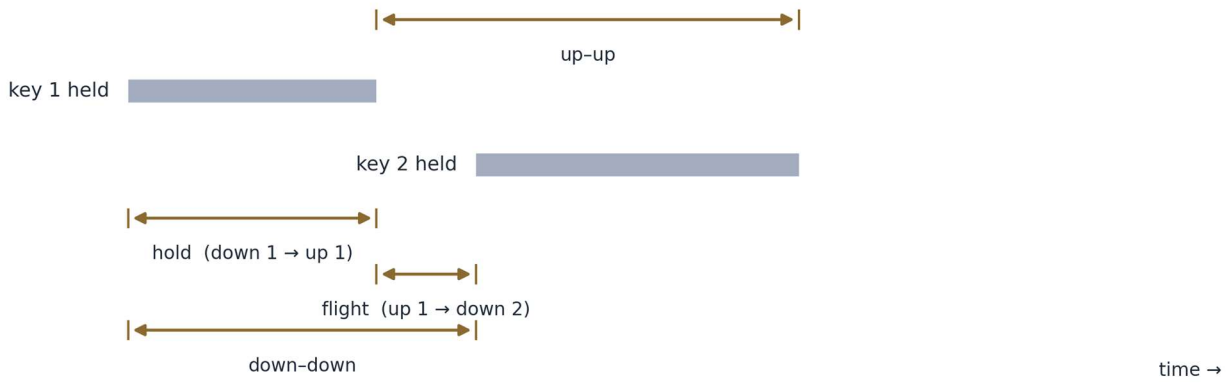


One typing sample flows left to right: raw keystrokes become timing features, the encoder reduces them to a 128-number fingerprint, and the verifier compares that fingerprint with the user's enrolled cluster to reach a decision. Enrolment follows the same path but stores the cluster instead of judging against it.

The data. Every reported number comes from the public CMU benchmark: 51 people each typed the same 11-character password 400 times, giving 20,400 complete samples. One sample is one full entry of the password, eleven keystrokes. I kept my own platform's users out of the reported experiments and trained only on this public, anonymised set.

What the system measures. Each keystroke contributes four timings: how long the key is held, the gap from one press to the next, the gap from a release to the next press, and the gap between releases. One sample is the full 11-keystroke password, so the model reads a short sequence of these numbers, paired with which key was struck. The timings stay raw, in seconds, with no normalisation. The rhythm itself is the signal. The same feature code runs in training and in the live service, so the deployed model can never be fed input prepared differently from what it learned on.

Figure 2: the four timings, drawn on two keystrokes.



Hold is how long a key stays down; flight is the silence between one key's release and the next key's press. Flight can be negative when a fast typist overlaps keys, and the model receives it unclamped.

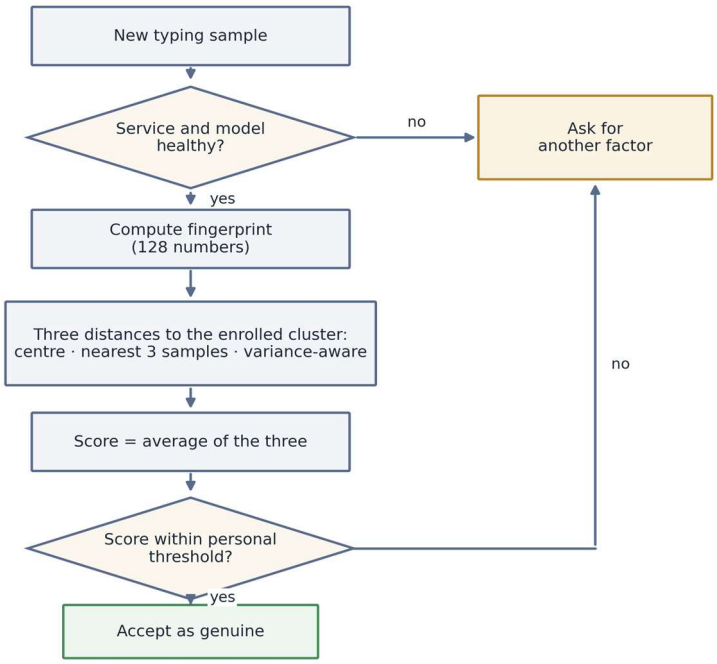
Imagine a space in which every burst of typing becomes a single point. A well-trained network places the same person's points in a tight cluster and different people's far apart, so recognising someone becomes a matter of distance: does this new point fall inside your cluster, or closer to someone else's? My network reduces each burst to 128 numbers, its position in that space. I trained it the way face-recognition systems are trained [3]: show it two samples from one person and one from another, adjust it to pull the pair together and push the third away, and repeat hundreds of thousands of times until the clustering emerges. The full design is in Appendix C; it trains on a laptop in about 23 minutes and fingerprints a sample in under a millisecond.

Inside the encoder. A convolutional layer reads short local patterns in the timing. The recurrent layer that follows reads the whole sequence in order, forwards and backwards, so context on both sides of a keystroke counts. Attention then weighs which keystrokes carried the most identity, and the result settles into the 128 numbers. The whole encoder is 83,505 parameters, around a third of a megabyte, sized for how little an 11-key password can teach.

Training, in brief. Sixty passes over the 35 training subjects, in batches holding two samples from each of 16 people, so every batch contains matching and non-matching pairs. Within each batch the training works on the hardest cases present, the same-person pair that landed farthest apart and the different-person pair that landed closest, and a second, gentler pull keeps each person's samples near their own centre.

To decide whether a new sample is genuine, I measure how far it falls from the enrolled cluster, combining three distances: to the cluster's centre, to the nearest few enrolled samples, and a third that allows for how much the person's typing naturally varies. Enrolling takes about a dozen samples and no retraining; the threshold comes from how consistent those samples are. Each sample is scored against the others and the bar sits just above what that person's own typing normally scores, so the threshold widens or tightens with their natural consistency.

Figure 3: the verification decision.



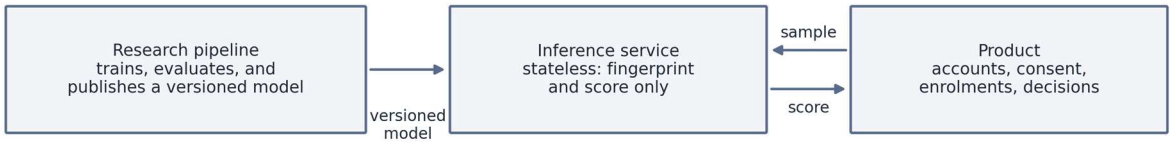
Every check ends in one of two outcomes: the sample is accepted as genuine, or the service asks for another factor. No failure path admits the user.

The decision logic, as pseudocode.

```
ENROL(user):
    samples   <- about 12 consented typing samples
    F         <- fingerprint(s) for each sample s        # 128 numbers each
    centroid  <- mean(F)
    C         <- covariance of F, shrunk toward a stable target
    threshold <- 90th percentile of leave-one-out genuine distances x 1.15
    threshold <- max(threshold, 0.001)                   # floor from the live crash
    store centroid, F, C, threshold

VERIFY(user, sample):
    if service or model checks fail:  return ASK_ANOTHER_FACTOR
    f  <- fingerprint(sample)
    d1 <- L1 distance from f to centroid
    d2 <- mean L1 distance to the 3 nearest enrolled fingerprints
    d3 <- Mahalanobis distance under C
    if C is unusable: score <- d1   else: score <- mean(d1, d2, d3)
    if score <= threshold:  return GENUINE
    else:                   return ASK_ANOTHER_FACTOR
```

Research, serving, product. The architecture separates three concerns: a research pipeline that trains and evaluates; a stateless inference service that turns samples into fingerprints and scores them; and the product, which holds accounts, consent records and enrolments. The inference service stores nothing about any user. The product checks the model version on every call and treats a mismatch as failure, and failure asks for another factor.

Figure 4: the three-part architecture.



checks the model version on every call;
a mismatch is a failure, and failure asks for another factor

Training never touches the live service, and the live service never touches user records. Each part can change or fail without taking the others with it.

I built it into an application: the user consents, types a few times to enrol, and is verified on a new sample. On any failure the service requests another factor; it never admits by default. In a live check with one held-out person, genuine samples averaged 3.10 and impostors 6.73 (lower means more genuine), so the deployed system does rank impostors below the real user. Its first run also failed, in a revealing way I return to below.

4. Evaluation design

My first evaluation looked strong, until I realised I had trained and tested on the same 51 subjects, textbook data leakage that made the score worthless.

I split the 51 people into 35 for training and 16 held out, and added a runtime check that makes it impossible for a test subject to leak into training. Judged only on those 16 strangers, the true error rate proved far higher.

The guard is not a habit or a promise; it is these lines, verbatim, from the training script:

```
# --- OPEN-SET split: train on train-subjects, evaluate on held-out subjects.
train_subjects, test_subjects = split_subjects(subjects, args.seed)
train_set = set(train_subjects)
enc, history = train_on_subjects(by_subject, train_subjects, args)

# Leakage guard: NO test subject may appear in the training set.
assert not (set(test_subjects) & train_set), \
    "open-set violation: a test subject leaked into the training set"
```

The most useful bugs were never the ones a test caught, but the ones I found by asking whether a passing test proved what it claimed. Three stand out. The live service crashed for one unusually consistent typist, whose near-identical passwords drove their personal threshold close to zero and overflowed a later calculation. My artificially varied test data could never have triggered that. Only a real person did. On the first day, the real data file trained on all-zero timings, because its columns were named differently from my test fixture and my code read straight past them while reporting a plausible number. And for a while the blended decision-maker I was most pleased with was measured nowhere. The evaluation scored only the simple distance, so the ensemble ran in the live product while earning no result. I noticed this only when I asked why my supposedly better method had no figure, and wiring it into the evaluation is where the 10.2% comes from. The full log of all thirteen fixes is in the project repository.

The crash fix lives in the service today, carrying the comment that records who found it and how:

```
def _confidence(score, threshold, k=5.0):
    if threshold <= 0:
        return 100.0 if score <= 0.1 else 0.0
    # Clamp the exponent argument before math.exp. A very consistent typist on a
    # short fixed password yields a near-zero threshold, so score/threshold can be
    # huge and math.exp would raise OverflowError (HTTP 500). The sigmoid
    # saturates well before +/-60, so clamping there is loss-free for the output
    # but removes the crash. (Found via the live CMU end-to-end test, subject
    # s005; see research/artifacts/problem_log.json: confidence-overflow.)
    expo = k * (score / threshold - 1.0)
    expo = max(-60.0, min(60.0, expo))
    val = 100.0 / (1.0 + math.exp(expo))
    return round(min(100.0, max(0.0, val)), 2)
```

5. Results

The headline result, measured on the 16 strangers and averaged over three runs, is below. Lower is better.

Decision method	Error rate	Compared with
Simple Distance	**14.2%**	the 2009 benchmark's 9.6%
Full Blended Method	**10.2%**	(no direct baseline)

The system recognises strangers far better than chance, live and end to end, but on this small fixed-password test it does not beat the 2009 hand-built method, scoring 14.2% against 9.6%. That headline figure is the balanced point of the error trade-off; Figure B.1 gives the full curve from the strictest setting to the most lenient.

The blended method, at 10.2%, is both more accurate than the simple one and far steadier, barely moving between runs. Over 14 random splits it won every time, which is very unlikely to be chance. That steadiness comes almost entirely from the most sophisticated of the three distance measures, and it points to a clear improvement. The headline runs sit on the favourable side: averaged across all 14 splits the two methods score about 18.7% and 13.3%, so the true gap to 9.6% is wider than the headline suggests.

Individuals differ enormously. The easiest person to recognise had an error rate under 1%, the hardest over 33%, so on the same model and the same password one person is thirty-seven times harder to identify than another. The difference is not random. The system struggles with people whose typing is so inconsistent that it overlaps with everyone else's, which means the real ceiling is the data: an 11-key password holds little identifying information.

Figure B.2 shows this directly. Reduced to two dimensions, several people's 128-number fingerprints form tight, clearly separated clusters despite the model never training on them, while a blurred central region holds the hard-to-recognise group. More training and more samples per person did not help; the model's internal measure showed it had already learned everything the short password can teach. The experiments behind this are in Appendix C.

The build held up on most of what I set out to do. The fingerprint works and runs in under a millisecond on an ordinary laptop, and it clusters strangers it never trained on, as Figure B.2 shows. The score is measured open-set on 16 unseen people, behind a guard that makes a leak impossible. The simple and blended scorers come from a single run, so the comparison is fair rather than selective, and the whole system re-runs from one command on a pinned dataset and fails safe when it breaks. The one aim it did not meet was the headline: to beat the best published result. I came close and fell short.

6. Discussion

Typing rhythm identifies a person only in part, but well enough to serve as a useful second check behind a password. An attacker who holds your stolen password must still type as you do. Run continuously, the system could even detect that the person mid-session had changed. It does not beat a good hand-built method on small, fixed-text data. The more useful finding is narrower. Placing classical statistics on top of a learned fingerprint makes the decision more stable, and I can name the component responsible. That is the contribution: a reproducible measurement that a classical verifier run inside a learned fingerprint is steadier than either half alone, with that component identified and every failure mode recorded. On a small public benchmark, tested only on strangers, that measurement did not exist before.

My 14.2% sits above the 2009 method's 9.6% on its own ground and far short of TypeNet's 2.2%, though that 2.2% is bought with internet-scale data I do not have. Under identical conditions I also tested a Transformer, the architecture behind most of today's best-known AI models, and it did clearly worse and less reliably, at 19.8%. On data this small, my simpler design's built-in assumptions about timing and sequence beat the Transformer's flexibility, which is a concrete reason for the choice.

The limitations are real. The dataset is a single small one, the password a single 11-key phrase, and the model small enough to run on a laptop. The system is tested on fixed text only: the free-typing version is built but not yet measured on a large corpus. The confidence percentage it reports is not yet calibrated, though the accept-or-reject ordering behind it is sound. None of this makes the result wrong, but all of it is reason not to overstate it.

7. Ethics and responsible use

Anything that can recognise people by their behaviour can also monitor them, so I built the ethics into the design from the start.

- **Typing data is legally among the most protected kinds.** Under GDPR [11], biometric data used to identify someone is "special category" data, and the law explicitly includes behavioural traits. Once this system identifies a user, it is handling data of that class, which shaped every decision below.
- **Public, anonymous data only.** The headline result is measured entirely on the 2009 benchmark, whose subjects are anonymised as s002, s005, and so on, consented to research use, and published by its authors for exactly this purpose. I deliberately did not train the reported model on real users of my product, which would have created fresh identifiable behavioural data, with all the consent and storage duties that brings, for no scientific gain.

- **Consent is opt-in and revocable, and I store the fingerprint, not the typing.** Opting out deletes the profile. The service keeps no record of what was typed, only the derived fingerprint and a log of decisions. A stolen fingerprint is far less damaging than a recording of everything a user wrote.
- **It fails safe, never open.** Any failure forces another factor and never defaults to admitting the user.
- **It fails some people far more than others, which is a fairness problem.** My 37-fold spread in error rate is a clear disparate-impact risk: the people it serves worst would be wrongly rejected far more often than the 14.2% average admits, and the average hides this entirely. It must therefore never be the only check, and a real deployment would need a per-person error audit rather than a single average. Those people can be identified in advance, since they are the least consistent typists, so a fair system can flag them and fall back to another factor instead of quietly failing them.
- **The same technology in the wrong hands.** What protects an account could equally track or de-anonymise people by their typing. I present it as opt-in protection the user controls, report the error rates openly, and would never present a system with a 10% error rate as infallible.

8. Reflection and future work

The most important thing I learned was to distrust my own results. My first evaluation produced a number I was delighted with, and it was worthless. Catching that taught me to ask "what would make this wrong?" before trusting any good result. It also taught me why biometrics are judged on error-rate curves rather than plain accuracy: when a system can fail in two opposite ways, one reassuring number had hidden the trade-off that mattered.

Working without a mentor had costs as well as benefits. Every check on my work was one I ran myself, which is probably why I caught the flaw at all, having learned to doubt my own code. It is also where working alone cost me: there was no one to ask whether those were my own training subjects, so the mistake stood for a while with only me to catch it. Next time I would write the whole evaluation procedure down before writing any code for it, since on paper the leak would have been obvious, and I would find one person allowed to doubt me early.

Where I would take it next follows from the ceiling being the *data* rather than the model:

- **Free typing on a large real dataset**, the 136-million-keystroke Aalto corpus [7]. The pipeline is built and needs only the data.
- **Calibrate the confidence scale** so the percentage the product displays is meaningful.
- **Train at scale** on appropriate hardware, in the manner of TypeNet, to test whether the hybrid's advantage holds as accuracy improves.

- **Collect a small, consented set of real users** to test whether it generalises across datasets, with the same ethical safeguards built in from the outset.

I set out to find whether the way a person types could be a quiet second lock on their identity. It can, though not a perfect one and not yet better than the 2009 benchmark on a test this small. What I built is a real one, measured and reproducible.

Acknowledgements

This was an independent project with no mentor, and the research community deserves a lot of credit: Killourhy and Maxion for the dataset and benchmark, FaceNet and TypeNet for the methods, Ledoit and Wolf for the statistical method, and the maintainers of PyTorch, NumPy, scikit-learn and FastAPI. The international biometric-testing standard and the UK data-protection guidance shaped my testing and ethics. A beginner AI course covering machine learning gave me the grounding to start, and the documentation and community forums of the libraries above stood in for the colleagues a lab would have provided. I would also like to thank everyone who helped me complete this project, especially my parents.

A Note on AI use

I used Anthropic's Claude, through the Claude Code assistant, across the whole project, from November 2025 to July 2026, and the commit history holds the dated trail. I had taken a beginner AI course covering machine learning, so I knew the basics of what I was building. I used Claude to brainstorm and think through ideas, and as a coding aid: it drafted some functions, which I reviewed, ran and tested, and it helped me track down bugs.

References

1. Killourhy, K. S. & Maxion, R. A. (2009). *Comparing Anomaly-Detection Algorithms for Keystroke Dynamics*. DSN-2009, pp. 125–134. Dataset: https://www.cs.cmu.edu/~keystroke/
2. Acien, A., Morales, A., Monaco, J. V., Vera-Rodriguez, R. & Fierrez, J. (2021). *TypeNet: Deep Learning Keystroke Biometrics*. IEEE T-BIOM. arXiv:2101.05570.
3. Schroff, F., Kalenichenko, D. & Philbin, J. (2015). *FaceNet: A Unified Embedding for Face Recognition and Clustering*. CVPR 2015. arXiv:1503.03832.
4. Hermans, A., Beyer, L. & Leibe, B. (2017). *In Defense of the Triplet Loss for Person Re-Identification*. arXiv:1703.07737.
5. Wen, Y., Zhang, K., Li, Z. & Qiao, Y. (2016). *A Discriminative Feature Learning Approach for Deep Face Recognition (center loss)*. ECCV 2016.
6. Ledoit, O. & Wolf, M. (2004). *A Well-Conditioned Estimator for Large-Dimensional Covariance Matrices*. J. Multivariate Analysis 88(2), 365–411.
7. Dhakal, V., Feit, A. M., Kristensson, P. O. & Oulasvirta, A. (2018). *Observations on Typing from 136 Million Keystrokes (Aalto dataset)*. CHI 2018. Data: https://userinterfaces.aalto.fi/136Mkeystrokes/
8. ISO/IEC 19795-1. *Biometric performance testing and reporting — Part 1*. https://www.iso.org/standard/73515.html

9. *Keystroke Dynamics: Concepts, Techniques, and Applications*. ACM Computing Surveys (2024/25). DOI 10.1145/3733103.
10. Frank, M. et al. (2013). *Touchalytics: Touchscreen Input as a Behavioral Biometric for Continuous Authentication*. IEEE TIFS 8(1). arXiv:1207.6231.
11. UK GDPR, Article 9 (special-category data) and Article 4(14) (biometric data). https://gdpr-info.eu/art-9-gdpr/
12. Verizon (2024). *2024 Data Breach Investigations Report*. https://www.verizon.com/business/resources/reports/dbir/
13. Cho, K. et al. (2014). *Learning Phrase Representations using RNN Encoder–Decoder (GRU)*. EMNLP 2014. arXiv:1406.1078.
14. Bahdanau, D., Cho, K. & Bengio, Y. (2015). *Neural Machine Translation by Jointly Learning to Align and Translate (attention)*. ICLR 2015. arXiv:1409.0473.
15. van der Maaten, L. & Hinton, G. (2008). *Visualizing Data using t-SNE*. JMLR 9, 2579–2605.

Appendices

Appendix A: Glossary

- **Fingerprint / embedding.** The 128 numbers the model produces for one typing sample; same-person fingerprints land close together.
- **Equal error rate (EER).** The balanced setting where "impostor gets in" and "real user locked out" are equally likely; 10% ≈ wrong one time in ten.
- **Open-set test.** Judged only on people not seen in training; the correct test for authentication.
- **Mahalanobis / Ledoit–Wolf.** A distance measure that accounts for how much a person's own typing naturally varies, kept stable when enrolment samples are few.

Appendix B: Full results, figures and provenance

Per-subject error rate (simple distance, seed 42), 16 held-out people, most to least distinctive:

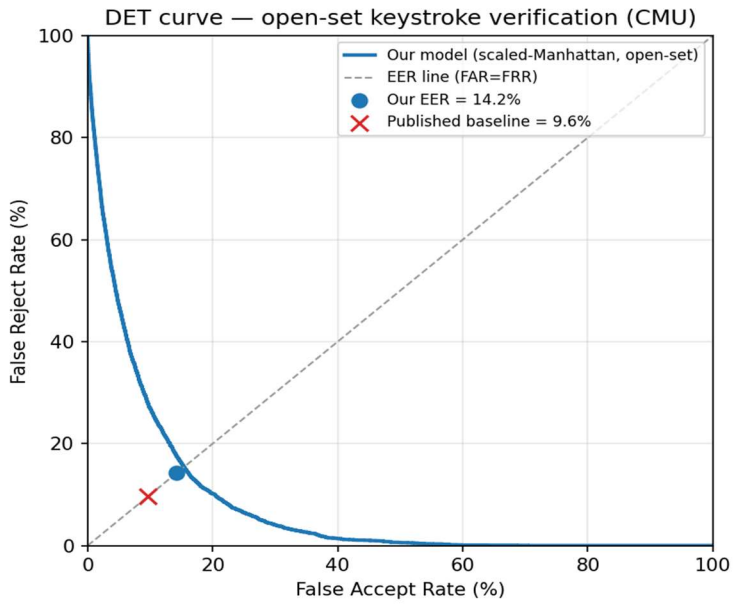
Rank	Subject	EER	Rank	Subject	EER
1	s036	0.009	9	s050	0.179
2	s017	0.020	10	s056	0.190
3	s022	0.035	11	s030	0.190
4	s005	0.045	12	s018	0.205
5	s012	0.051	13	s054	0.215
6	s010	0.090	14	s037	0.234
7	s038	0.090	15	s007	0.295
8	s053	0.090	16	s047	0.335

Average of these is 0.1421, matching the headline seed-42 figure. Aggregate over the three runs (seeds 42/43/44): simple distance 0.1422 ± 0.0279; full blend 0.1016 ± 0.0097; benchmark 0.0962. The 0.9%–33.5% spread is the fairness finding.

Provenance (for independent verification).

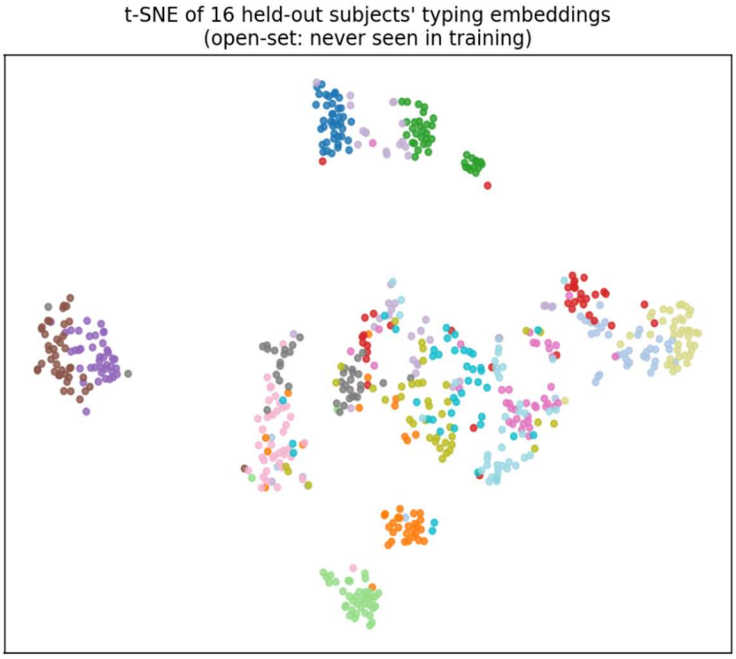
- Dataset (`DSL-StrongPasswordData.csv`) SHA-256: `b11d23538b1865fa6ecf4e8b78567caa312e9c1027604bb022fcc6ad7eaa7a33`
- Git commit of the recorded run: `13fafe8f039d969fd77734b67d2457d37c59f918`
- Code, data pipeline and the full fix log: https://github.com/devadit1515/typing-sanctuary
- One-command rerun, from the repository's research directory: pwsh -File scripts/reproduce.ps1, which verifies the dataset hash, trains all three seeds, asserts the error rates and regenerates both figures.
- Seeds 42 / 43 / 44 · 128-number fingerprint · 60 training passes · laptop CPU · ~23 min · Python 3.12, PyTorch 2.5.1 (CPU). All versions pinned in `research/requirements.txt` (NumPy 2.1.3), so "the same environment" is a checkable claim, not a promise
- All runs on my own laptop: Intel Core Ultra 9 285H CPU, 32 GB RAM, no GPU used.

Figure B.1: the error trade-off (DET curve).



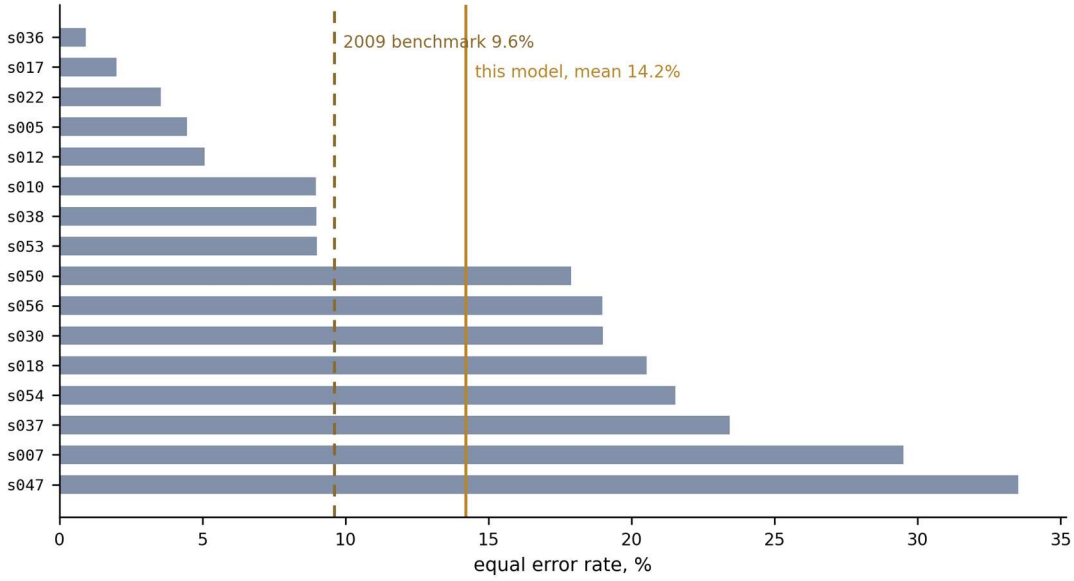
The trade-off between locking out genuine users and letting impostors in, across every dial setting, for the simple-distance method on the 16 strangers. The balanced (equal-error) point is at 14.2%; the 9.6% benchmark is marked.

Figure B.2: the fingerprints, as a picture (t-SNE).



Each person's 128-number fingerprints, squashed to a 2-D picture and coloured by person. Several people form tight, well-separated clusters though the model never trained on them; the muddy middle matches the hard-to-recognise people.

Figure B.3: per-subject error rates (simple distance, seed 42).



One bar per held-out person. The spread between the easiest and the hardest, 0.9% to 33.5%, is the fairness finding.

Appendix C: Technical specification

The exact designs and numbers, for readers who want them. None of it is needed to follow the report.

Features. One window = one complete password entry (11 keystrokes; 51 subjects × 400 repetitions = 20,400 windows). Each keystroke = four timing features (hold, down–down, flight, up–up) plus a 16-dimensional learned character embedding (a 20-D per-keystroke vector). Timings are the dataset's raw values in seconds, differences only, shifted to a per-window origin (the first keystroke's down-time) purely to avoid float32 cancellation at large timestamp magnitudes; there is **no** per-feature normalisation or z-scoring, and negative flight times from overlapping keys pass through unclamped. The identical featurisation code runs in training and serving, so the deployed model cannot see different features from the trained one.

Encoder (`KeystrokeEncoder`, 83,505 parameters, 0.32 MB float32). Input fusion (20-D) → two `Conv1d` layers (20→64→64, kernel 3) → bidirectional GRU (64 each way → 128; Cho et al., 2014) → single-head additive attention with a length mask (Bahdanau et al., 2015) → linear projection to 128-D → L2-normalisation. Embeds one window in ≈ 0.8 ms (mean of 200 runs, batch of one).

Training. Batch-hard triplet loss (margin 0.2; Hermans et al., 2017) + center loss (weight 0.01; Wen et al., 2016), Adam (lr 1e-3; Kingma & Ba, 2015), 60 epochs, batches of 16 subjects × 2 windows = 32, squared-Euclidean distance on L2-normalised embeddings. Deterministic: same seed → same weights bit-for-bit on CPU.

Verification ensemble. Enrolment stores the centroid, the enrolment embeddings (k-NN, k = 3), and the Ledoit–Wolf inverse-covariance matrix. Score = unweighted mean of (i) L1 distance to the centroid, (ii) mean L1 distance to the 3 nearest enrolment embeddings, (iii) Ledoit–Wolf Mahalanobis distance; singular covariance falls back to the centroid distance. Per-user threshold = 90th percentile of leave-one-out genuine distances × 1.15, floored at 10^{-3}, the exact floor whose absence caused the live crash. Note the two enrolment regimes: the *benchmark* enrols with 200 windows (half a subject's data) to measure what the representation can do; the *product* enrols with about a dozen, which is workable only because the leave-one-out threshold adapts to however consistent that user's dozen turns out to be.

Why these pieces work: the mechanism, not just the name. *L2-normalisation divides each fingerprint by its own length, placing it on a unit sphere, so distance depends on the direction of the rhythm pattern rather than its magnitude, its shape rather than how fast someone types. That is what matters when comparing patterns. Batch-hard triplet loss trains on triples of two same-person samples and one from someone else. For each anchor it finds the hardest pair in the mini-batch, the same-person sample that landed farthest away and the different-person sample that landed closest, and pushes them apart by a fixed margin, so training spends its effort on the genuinely confusable cases rather than the easy ones. Ledoit–Wolf shrinkage [6] solves a small-sample problem. With only*

about 12 enrolment samples in 128 dimensions, the covariance matrix a Mahalanobis distance needs is highly unstable and often not invertible, so I blend it toward a well-behaved target, just enough to guarantee a stable, invertible matrix. This is why the Mahalanobis term carries the ensemble: it is the only one of the three distances that captures how a person's typing dimensions co-vary, not merely how far each sits from their average.

Evaluation. Open-set 35/16 subject split (seeded). Per test subject, the *earlier* 200 of their 400 windows enrol and the *later* 200 are tested: a positional split, chosen deliberately so enrolment never sees the future (mildly pessimistic under session drift, never flattering). Impostors = all 400 windows of each of the other 15 test subjects (6,000 impostor scores per subject). The evaluator raises an error rather than invent a number if a test set would be empty. The headline is the *mean of per-subject EERs*, matching the 2009 baseline's methodology, not the pooled-score EER, which mixes subjects with different score scales (pooled, seed 42: 16.1%). Two scorers (scaled-Manhattan headline; full ensemble secondary), never cross-compared. Seeds 42/43/44 for the headline; 14 seeds for the ensemble-vs-primary comparison, where the ensemble wins 14/14 (Wilcoxon signed-rank $p \approx 0.0001$). Subject-level bootstrap (20,000 draws) gives a 95% CI of [9.5%, 19.2%] on the seed-42 14.2%. Separability (nearest-impostor distance ÷ within-subject scatter) vs. per-subject EER: Spearman $\rho = -0.84$, $p = 0.0001$. 14-seed means: primary ≈ 18.7%, ensemble ≈ 13.3%.

Operating points (because a real deployment must pick a dial setting, not sit at the balanced point). On the pooled seed-42 score distribution (one global threshold, so these are *more pessimistic* than the per-user thresholds actually deployed), tolerating a 5% lock-out rate for genuine users admits ≈18.5% of impostor attempts; tolerating 10% lock-out admits ≈12.4%. This is the security-vs-convenience dial in numbers, and it is why this system is a second factor, never the only lock.

Hyperparameter search (nested validation). Test 16 held out; inside the 35, a 24-inner-train / 11-validation split; one setting changed at a time, judged only on the 11:

Change from original	Validation EER (primary / ensemble)
Original (60 epochs, margin 0.2, centre 0.01, 2 windows/subj)	0.1933 / 0.1678
more windows/subj (2→4)	0.2572 / 0.1731
more windows/subj (2→8)	0.1975 / 0.2023
longer training (60→120 epochs)	0.1904 / 0.1684
wider margin (0.2→0.3)	0.1933 / 0.1678
stronger centre-loss (0.01→0.05)	0.2054 / 0.1725

The validation “winner” (120 epochs) scored 0.1610 ± 0.0866 / 0.1128 ± 0.0378 on the real test, worse and less stable than the original’s 0.1422 ± 0.0279 / 0.1016 ± 0.0097, with one seed at 0.284. Training loss saturates at the margin in every row: the bottleneck is the 11-key password, not the optimisation.

Which distance carries the ensemble. Mean over 3 seeds: full 0.1016; drop centroid-L1 0.1016; drop k-NN 0.1016; drop Mahalanobis 0.1330. Over 14 seeds: full 0.1326; drop Mahalanobis 0.1783; drop either other term 0.1326. The Ledoit–Wolf Mahalanobis term carries essentially all of the advantage; the other two are numerically swamped by its larger scale. Rescaling the three before blending would let the other two contribute.

Transformer comparison. A comparable Transformer encoder (2 layers, 4 heads, 77,296 parameters, slightly *smaller* than the CNN+BiGRU), identical protocol, same 3 seeds: primary EER 19.8% ± 5.4%, ensemble 12.3% ± 4.9%, against this model’s 14.2% ± 2.8% and 10.2% ± 1.0%. Both models’ loss saturates at the margin, so both are data-limited; the convolution/recurrence priors extract a steadier signal from small data than self-attention.

Appendix D: Selected code

Batch-hard mining. From research/ksbio/triplet.py. For every sample in a batch, find the worst-placed pair and train on exactly that.

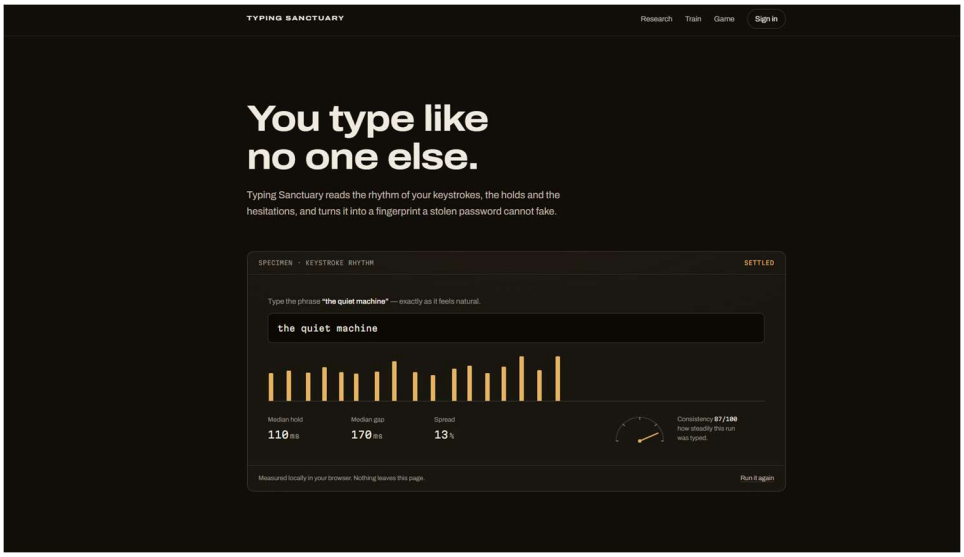
```
def batch_hard_triplet_loss(emb, labels, margin=0.2):
    d = _pairwise_sq_dists(emb)                       # [b,b]
    same = labels.unsqueeze(0) == labels.unsqueeze(1) # [b,b] bool
    eye = torch.eye(len(labels), dtype=torch.bool, device=emb.device)
    pos_mask = same & ~eye
    neg_mask = ~same
    # hardest positive: max distance among same-class (0 if none)
    pos_d = (d * pos_mask).max(dim=1).values
    # hardest negative: min distance among other-class (inf if none)
    neg_d = d.masked_fill(~neg_mask, float("inf")).min(dim=1).values
    valid = pos_mask.any(dim=1) & neg_mask.any(dim=1)
    losses = F.relu(pos_d - neg_d + margin)
    losses = losses[valid]
    return losses.mean() if losses.numel() else emb.sum() * 0.0
```

The ensemble decision. From ml-service/app/verify.py - the code run on every check.

```
def verify(embedding, profile):
    centroid = profile["centroid"]
    refs = profile["refs"]
    threshold = profile["threshold"]
    cov_inv = profile.get("covInverse")

    manhattan = _l1_to_centroid(embedding, centroid)
    knn = _knn(embedding, refs)
    maha = _mahalanobis(embedding, centroid, cov_inv) if cov_inv else manhattan

    components = {"manhattan": manhattan, "knn": knn, "maha": maha}
    score = sum(components.values()) / len(components)

    confidence = _confidence(score, threshold)
    risk = "LOW" if confidence >= 85 else "MEDIUM" if confidence >= 50 else "HIGH"
    return {"score": score, "confidence": confidence, "riskLevel": risk,
            "perComponent": components}
```

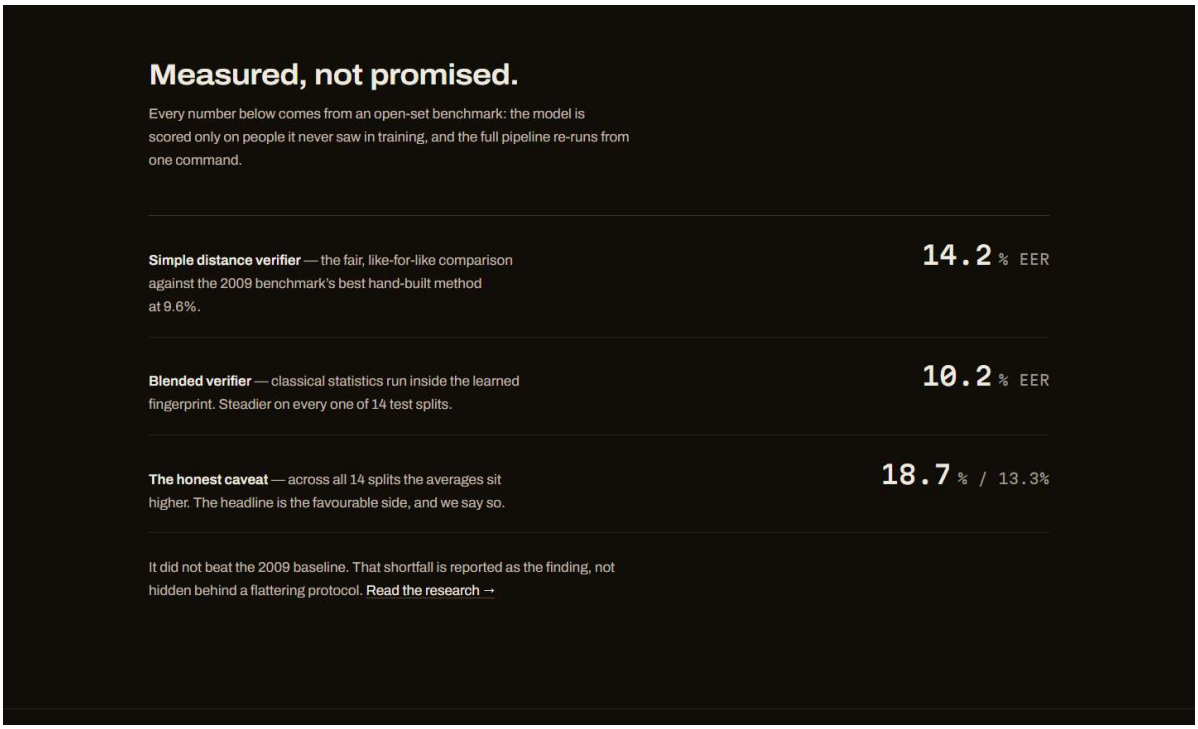
Appendix E: Website Pictures – deployed with render

Figure E.1: The landing page



Visitors type a short sample phrase and watch their own rhythm draw itself, one bar per keystroke, measured entirely in the browser.

Figure E.2: the results, stated on the site itself.

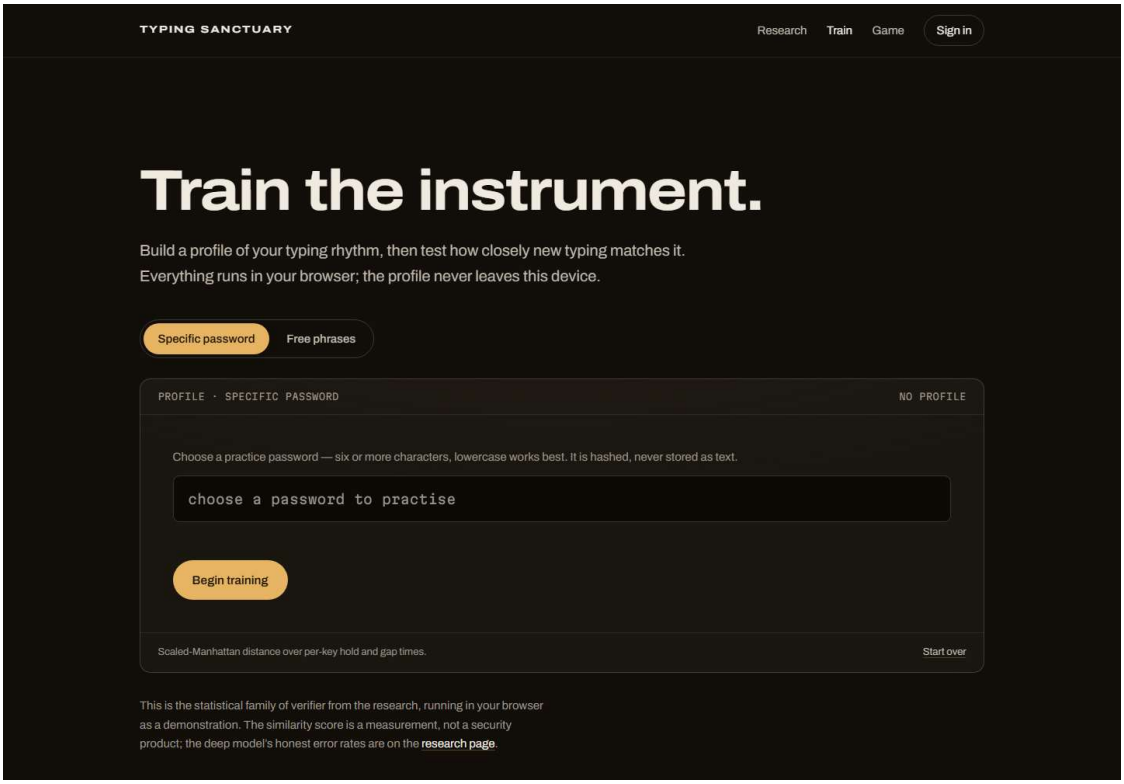


The same numbers this report gives, including the 14-split averages.

Figure E.3: the research page.

The result ledger with the confidence interval and the 37-fold fairness spread

Figure E.4: the public training instrument.



Visitors build a typing profile or train it on a specific password and can similarity

Figure E.5: Testing Similarity by Typing the Password it was trained on

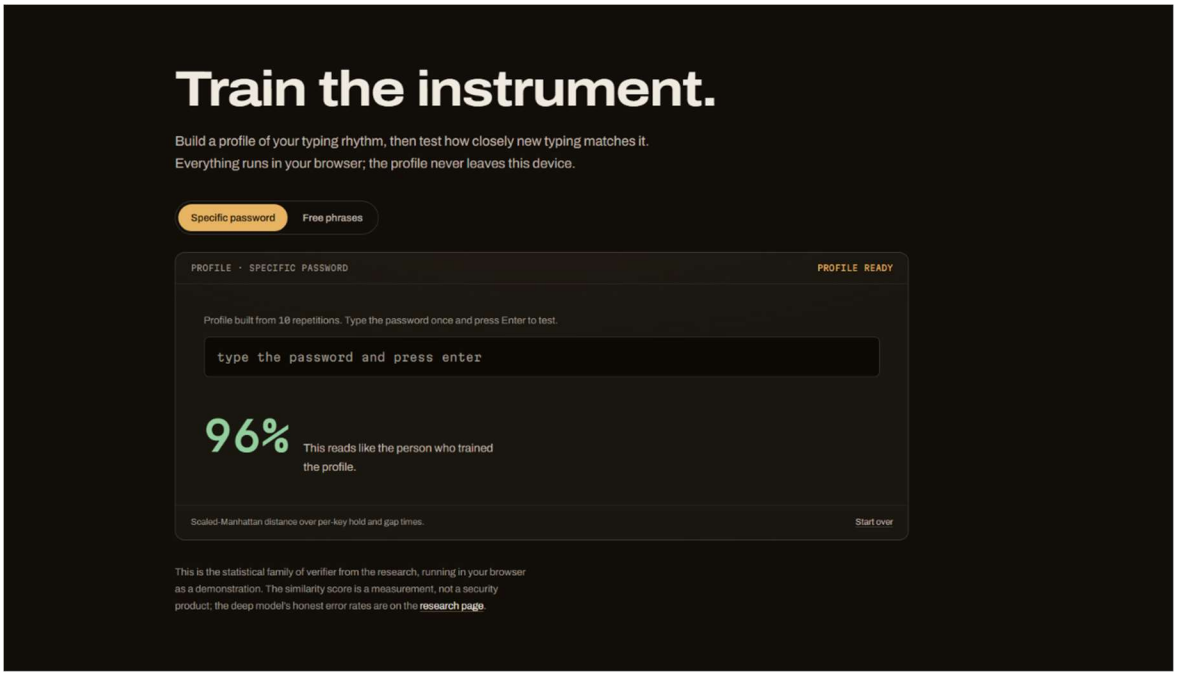


Figure E.6: Login Page (Mongo-DB used for the database)